🟢 SESIÓN 6 Relaciones JPA (Book ↔️ Category)

“Modelamos el dominio, no solo tablas.”

🟦 Bloque 1 — Punto de partida

Hasta ahora:

Book

- id (Long)
- title
- author
- category (String)

Problema real:

- La categoría no debería ser un String
- Varias books comparten la misma categoría
- No hay integridad referencial

🎯 Objetivos:

- Crear entidad Category
- Modelar relación @ManyToOne
- Añadir relación inversa @OneToMany
- Mantener DTO como contrato
- Entender cómo se persisten relaciones

🟦 Bloque 2 — Nueva entidad Category

📁 entity/**Category.java**

```
package com.cladsb1.books.entity;
import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "categories")
public class Category {

    @Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

/**
 * Nombre de categoría único (regla de negocio típica)
 */
@Column(nullable = false, unique = true, length = 60)
private String name;

/**
 * Relación inversa (bidireccional):
 * - mappedBy indica que la FK vive en Book.category
 * - IMPORTANTE: no expongas entities directamente para evitar bucles JSON.
 */
@OneToMany(mappedBy = "category", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Book> books = new ArrayList<>();

// JPA necesita constructor vacío
protected Category() {}

public Category(String name) {
    this.name = name;
}

// Getters
public Long getId() { return id; }
public String getName() { return name; }
public List<Book> getBooks() { return books; }

// Setter solo si lo necesitas
public void setName(String name) { this.name = name; }

    /**
     * Helpers para mantener consistencia bidireccional.
     * (opcional, pero buena práctica si gestionas la lista desde Category)
     */
    public void addBook(Book book) {
        books.add(book);
        book.setCategory(this);
    }

    public void removeBook(Book book) {
        books.remove(book);
        book.setCategory(null);
    }
}
```

🟦 Bloque 3 — Modificación de Book

📁 entity/**Book.java**

```
package com.cladsb1.books.entity;
import jakarta.persistence.*;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String title;

@Column(nullable = false, length = 80)
private String author;

/**
 * Muchos libros pertenecen a una categoría.
 * La FK se guarda en la tabla books (columna category_id).
 */
@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = "category_id", nullable = false)
private Category category;

// JPA necesita constructor vacío
protected Book() {}

public Book(String title, String author, Category category) {
    this.title = title;
    this.author = author;
    this.category = category;
}

    // Getters
    public Long getId() { return id; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public Category getCategory() { return category; }

    // Setters (JPA/PUT los suelen necesitar)
    public void setTitle(String title) { this.title = title; }
    public void setAuthor(String author) { this.author = author; }
    public void setCategory(Category category) { this.category = category; }

    /**
     * Setter para id (opcional). Útil si quieres forzar INSERT en POST: book.setId(null)
     */
    public void setId(Long id) { this.id = id; }
}
```

🧠 Teoría clave

@ManyToOne

FK en tabla books

@OneToMany(mappedBy="category")

lado inverso

Lado dueño

El lado dueño de la relación es @ManyToOne

Buena práctica

Nunca expongas entities directamente

🟦 Bloque 4 — Repository adicional

```
import com.cladsb1.booksdb.entity.Category;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface CategoryRepository extends JpaRepository<Category, Long> {

    Optional<Category> findByNameIgnoreCase(String name);
}
```

🟦 Bloque 5 — DTOs

Request

```
public record BookRequestDTO(
    @NotBlank String title,
    @NotBlank String author,
    @NotBlank String category
) {}
```

Response

```
public record BookResponseDTO(
 Long id,
 String title,
 String author,
 String category
) {}
```

🟦 Bloque 6 — Controller

(VERSIÓN CON STREAMS)

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.entity.Category;
import com.cladsb1.books.repository.BookRepository;
import com.cladsb1.books.repository.CategoryRepository;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookRepository bookRepository;
    private final CategoryRepository categoryRepository;

    public BookController(BookRepository bookRepository, CategoryRepository categoryRepository) {
        this.bookRepository = bookRepository;
        this.categoryRepository = categoryRepository;
    }

    //  READ all (200)
    @GetMapping
    public ResponseEntity<List<BookResponseDTO>> getAll() {
        List<BookResponseDTO> result = bookRepository.findAll()
                .stream()
                .map(this::toDTO)
                .toList();

        return ResponseEntity.ok(result);
    }

    //  READ by id (200 / 404)
    @GetMapping("/{id}")
    public ResponseEntity<BookResponseDTO> getById(@PathVariable Long id) {
        return bookRepository.findById(id)
                .map(this::toDTO)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    //  CREATE (201)
    @PostMapping
    public ResponseEntity<BookResponseDTO> create(@Valid @RequestBody BookRequestDTO dto) {

        Category category = categoryRepository
                .findByNameIgnoreCase(dto.category())
                .orElseGet(() -> categoryRepository.save(new Category(dto.category())));

        Book saved = bookRepository.save(new Book(dto.title(), dto.author(), category));

        URI location = URI.create("/api/books/" + saved.getId());
        return ResponseEntity.created(location).body(toDTO(saved));
    }

    //  UPDATE (200 / 404)
    @PutMapping("/{id}")
    public ResponseEntity<BookResponseDTO> update(@PathVariable Long id,
                                                  @Valid @RequestBody BookRequestDTO dto) {

        return bookRepository.findById(id)
                .map(existing -> {

                    Category category = categoryRepository
                            .findByNameIgnoreCase(dto.category())
                            .orElseGet(() -> categoryRepository.save(new Category(dto.category())));

                    existing.setTitle(dto.title());
                    existing.setAuthor(dto.author());
                    existing.setCategory(category);

                    Book saved = bookRepository.save(existing);
                    return ResponseEntity.ok(toDTO(saved));
                })
                .orElse(ResponseEntity.notFound().build());
    }

    //  DELETE (204 / 404)
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {

        if (!bookRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }

        bookRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    //  Mapping centralizado (Entity -> DTO)
    private BookResponseDTO toDTO(Book book) {
        return new BookResponseDTO(
                book.getId(),
                book.getTitle(),
                book.getAuthor(),
                book.getCategory().getName()
        );
    }
}
```

(VERSIÓN SIN STREAMS)

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.dto.response.BookResponseDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.entity.Category;
import com.cladsb1.books.repository.BookRepository;
import com.cladsb1.books.repository.CategoryRepository;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookRepository bookRepository;
    private final CategoryRepository categoryRepository;

    public BookController(BookRepository bookRepository, CategoryRepository categoryRepository) {
        this.bookRepository = bookRepository;
        this.categoryRepository = categoryRepository;
    }

    //  READ all (200)
    @GetMapping
    public ResponseEntity<List<BookResponseDTO>> getAll() {

        List<Book> books = bookRepository.findAll();
        List<BookResponseDTO> result = new ArrayList<>();

        for (Book b : books) {
            result.add(toDTO(b));
        }

        return ResponseEntity.ok(result);
    }

    //  READ by id (200 / 404)
    @GetMapping("/{id}")
    public ResponseEntity<BookResponseDTO> getById(@PathVariable Long id) {

        Optional<Book> opt = bookRepository.findById(id);
        if (opt.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.ok(toDTO(opt.get()));
    }

    //  CREATE (201)
    @PostMapping
    public ResponseEntity<BookResponseDTO> create(@Valid @RequestBody BookRequestDTO dto) {

        Optional<Category> optCategory = categoryRepository.findByNameIgnoreCase(dto.category());
        Category category;

        if (optCategory.isPresent()) {
            category = optCategory.get();
        } else {
            category = categoryRepository.save(new Category(dto.category()));
        }

        Book saved = bookRepository.save(new Book(dto.title(), dto.author(), category));

        URI location = URI.create("/api/books/" + saved.getId());
        return ResponseEntity.created(location).body(toDTO(saved));
    }

    //  UPDATE (200 / 404)
    @PutMapping("/{id}")
    public ResponseEntity<BookResponseDTO> update(@PathVariable Long id,
                                                  @Valid @RequestBody BookRequestDTO dto) {

        Optional<Book> optBook = bookRepository.findById(id);
        if (optBook.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        Book existing = optBook.get();

        Optional<Category> optCategory = categoryRepository.findByNameIgnoreCase(dto.category());
        Category category;

        if (optCategory.isPresent()) {
            category = optCategory.get();
        } else {
            category = categoryRepository.save(new Category(dto.category()));
        }

        existing.setTitle(dto.title());
        existing.setAuthor(dto.author());
        existing.setCategory(category);

        Book saved = bookRepository.save(existing);
        return ResponseEntity.ok(toDTO(saved));
    }

    //  DELETE (204 / 404)
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {

        boolean exists = bookRepository.existsById(id);
        if (!exists) {
            return ResponseEntity.notFound().build();
        }

        bookRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    //  Mapping centralizado (Entity -> DTO)
    private BookResponseDTO toDTO(Book book) {
        return new BookResponseDTO(
                book.getId(),
                book.getTitle(),
                book.getAuthor(),
                book.getCategory().getName()
        );
    }
}
```

🟦 Bloque 7— Tabla de comportamiento

Acción	Caso	Status
POST libro	categoría nueva	201
POST libro	categoría existente	201
GET libros	—	200
POST inválido	validación	400

🟦 Bloque 9 — Cierre conceptual

> "Las relaciones se modelan en el dominio. La API decide cómo exponerlas."

📎 ANEXO — PK MANUAL (2 formas)

🔵 Forma 1 — PK técnica autogenerada + campo único de negocio (RECOMENDADO)

Ejemplo: ISBN único.

```
@Column(nullable=false, unique=true)
private String isbn;
```

Ventajas:

- PK estable
- Buen rendimiento
- Buena práctica enterprise

🔵 Forma 2 — PK manual (Natural Key)

Ejemplo: Category con código como PK

```
@Entity
@Table(name="categories")
public class Category {
    @Id
    @Column(length=20)
    private String code; // lo manda el cliente

    @Column(nullable=false, unique=true)
    private String name;

    protected Category() {}

    public Category(String code, String name) {
        this.code = code;
        this.name = name;
    }
}
```

Repository:

```
JpaRepository<Category, String>
```

Consideraciones importantes

Si repites el mismo code:

- Puede hacer UPDATE
- O lanzar constraint violation

En proyectos reales:

- Se controla con existsById
- Se devuelve 409 Conflict

🔚 Cierre total de la sesión

Ahora sabemos:

- [ ] Modelar relaciones bidireccionales
- [ ] Entender quién es el lado dueño
- [ ] Evitar problemas JSON
- [ ] Diferenciar PK técnica vs natural key